Manual Técnico Completo – sis_doc (Servicios Docentes, CETIS 155)

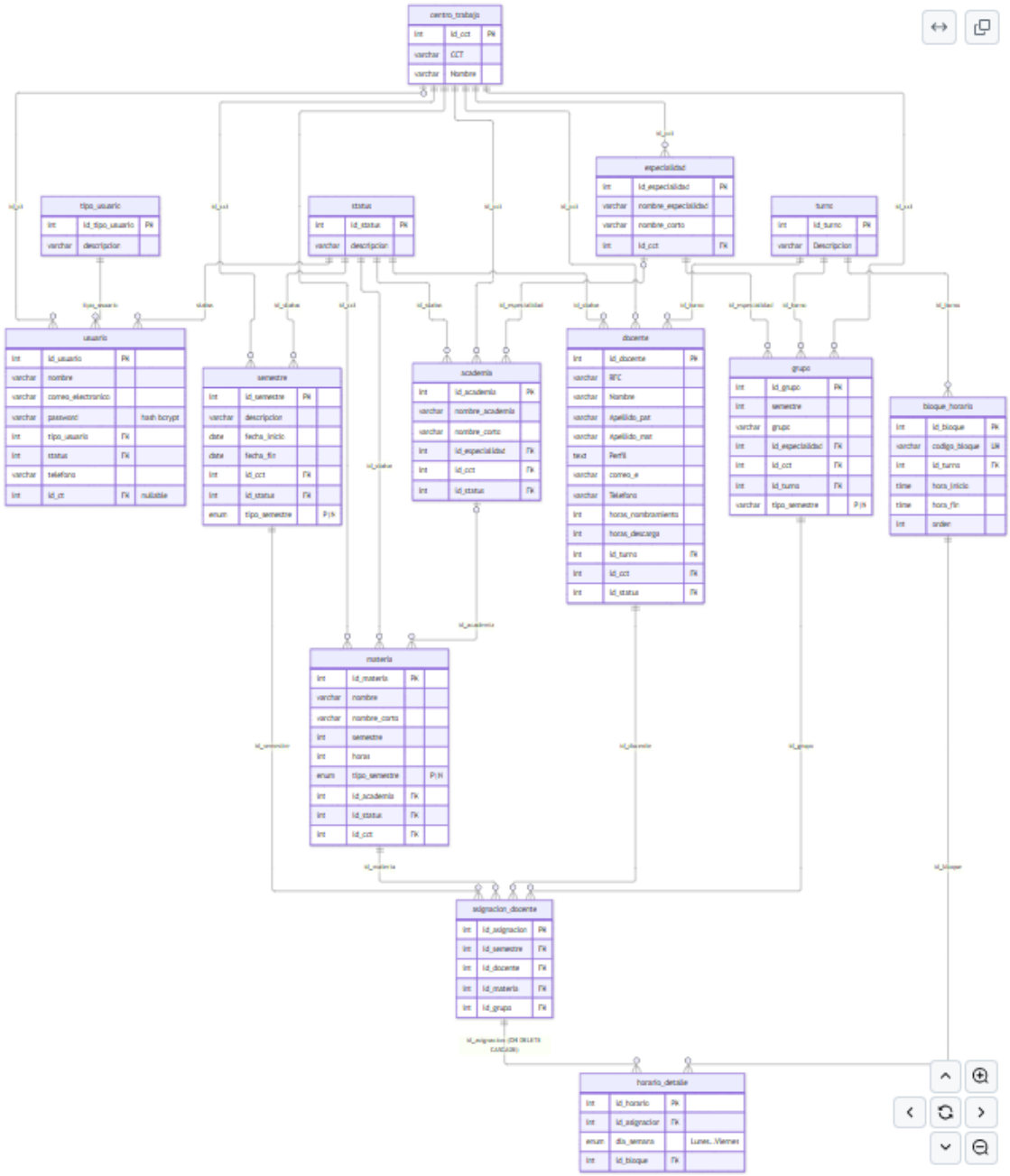
Documento generado a partir de sis_doc bd.sql, database/seed.sql, src/** y public/**.

---

1. Diagrama Entidad-Relación y Modelo de Datos (SQL)

Motor: MariaDB 10.4 / MySQL, InnoDB, utf8mb4_general_ci, base de datos sis_doc.

1.1 Diagrama ER



1.2 Tablas, claves y cardinalidad

Tabla	PK	Claves foráneas	Cardinalidad principal
centro_trabajo	id_cct	–	1 CCT → N docentes, materias, grupos, semestres, especialidades, academias, usuarios
status	id_status	–	1 status → N en docente/materia/semestre/academia/usuario (1 Activo, 2 Inactivo, 3 Suspendido, 4 Pendiente, 5 Baja)
tipo_usuario	id_tipo_usuario	–	1 tipo → N usuarios
turno	id_turno	–	1 Matutino, 2 Vespertino, 3 Mixto; 1 turno → N docentes, grupos, bloques
usuario	id_usuario	tipo_usuario, status, id_ct	N:1 con tipo, status y CCT (id_ct nulo = sin CCT)
especialidad	id_especialidad	id_cct	1 especialidad → N grupos y N academias
academia	id_academia	id_especialidad, id_cct, id_status	1 academia → N materias
docente	id_docente	id_turno, id_cct, id_status	1 docente → N asignaciones

Tabla	PK	Claves foráneas	Cardinalidad principal
materia	id_materia	id_academia, id_status, id_cct	1 materia → N asignaciones
grupo	id_grupo	id_especialidad, id_cct, id_turno	1 grupo → N asignaciones
semestre	id_semestre	id_cct, id_status	1 semestre → N asignaciones
asignacion_docente	id_asignacion	id_semestre, id_docente, id_materia, id_grupo	Resuelve la relación N:M docente–materia–grupo por semestre; 1 asignación → N celdas de horario
bloque_horario	id_bloque	id_turno	1 bloque → N celdas de horario
horario_detalle	id_horario	id_asignacion, id_bloque	Cada fila ocupa un (día, bloque) de una asignación

Bloques horarios sembrados: Matutino M1–M8 (07:00–14:00, receso antes de M4) y Vespertino V1–V8 (14:00–21:00, receso antes de V4).

1.3 Restricciones

ON DELETE CASCADE — única en todo el esquema:

- horario_detalle.id_asignacion → asignacion_docente.id_asignacion: al borrar una asignación se eliminan todas sus celdas de horario.

El resto de FKs usan el comportamiento por defecto (RESTRICT/NO ACTION), por lo que no se puede borrar un padre con hijos. Por eso la API usa **borrado lógico** (id_status = 5, "Baja") para docente, materia, academia y usuario, y para semestre establece id_status = 2 (Inactivo).

Claves UNIQUE:

Tabla	Índice	Columnas	Regla que garantiza
asignacion_docente	uk_asignacion_docente	(id_semestre, id_docente, id_materia, id_grupo)	No duplicar una asignación en el mismo semestre
horario_detalle	uk_asignacion_dia_bloque	(id_asignacion, dia_semana, id_bloque)	Una asignación no ocupa dos veces la misma celda
bloque_horario	codigo_bloque	codigo_bloque	Código de bloque único (M1…V8)

Otras restricciones: semestre.tipo_semestre y materia.tipo_semestre son ENUM('P','N'); horario_detalle.dia_semana es ENUM('Lunes','Martes','Miercoles','Jueves','Viernes') (sin acento; la API traduce Miércoles ↔ Miercoles).

Procedimiento almacenado: sp_insertar_grupo(p_semestre, p_grupo, p_id_especialidad, p_id_cct, p_id_turno) valida existencia de especialidad, CCT y turno (SIGNAL SQLSTATE '45000') y calcula tipo_semestre ('P' si el semestre es par, 'N' si es impar). La API actual inserta directamente en grupo y recibe tipo_semestre desde el cliente (validado a P/N).

Discrepancia de roles: el dump sis_doc bd.sql trae tipo_usuario = 1 Super Administrador, 2 Administrador, 3 Revisor, 4 Docente. database/seed.sql redefine 3 = Captura y agrega 5 = Revisor, que es lo que espera el código (ROLES en src/lib/auth.ts). Debe aplicarse el seed sobre el dump.

1.4 Vistas SQL de lectura

Vista	Tablas que une	Columnas clave expuestas	Uso en la API
vista_docente	docente ⋈ turno, centro_trabajo, status	nombre_completo (Apellido_pat Apellido_mat Nombre), turno, centro_trabajo, status	GET /docentes, catálogos
vista_materias	materia ⋈ academia, status	nombre_materia, nombre_academia, academia_corto, status_materia, id_cct	GET /materias, catálogos
vista_grupos	grupo ⋈ especialidad, centro_trabajo, turno (ORDER BY grupo)	especialidad_corta, CCT, centro_trabajo, turno	GET /grupos, catálogos
vista_semestre	semestre ⋈ centro_trabajo, status	descripcion_semestre, CCT, nombre_centro_trabajo, descripcion_status	GET /semestres, resolución de semestre activo
vista_academias	academia ⋈ centro_trabajo, status, especialidad	centro_trabajo, status_academia, nombre_especialidad	CRUD /academias, catálogos
vista_asignacion_docente	asignacion_docente ⋈ semestre, vista_docente, materia ⋈ grupo	nombre_docente, nombre_materia, horas_materia, grupo	GET /asignaciones

Vista	Tablas que une	Columnas clave expuestas	Uso en la API
		("<semestre>° <grupo>")	

horario_detalle no tiene vista; HORARIO_SELECT en api.ts hace el JOIN con asignacion_docente, docente, materia, grupo y bloque_horario.

---

2. Arquitectura del Código Fuente (TypeScript y frontend)

Stack: Node.js (ESM) + Express 4 + express-session + mysql2/promise + bcryptjs; TypeScript 5 ejecutado con tsx; frontend HTML/CSS/JS sin framework ni motor de plantillas. Scripts: npm run dev (tsx watch src/server.ts), npm run build (tsc), npm start, npm test.

2.1 Estructura de carpetas

sistema/

├── PLAN_DESARROLLO_COPILOT.md Reglas de negocio de referencia

├── package.json / tsconfig.json

├── sis_doc bd.sql Dump completo (esquema, datos, vistas, SP)

├── database/

| ├── schema.sql Esquema reproducible de la app

| └── seed.sql Catálogos base y roles (incluye Revisor = 5)

├── docs/

| └── MANUAL_TECNICO_COMPLETO.md Este documento

├── src/

| ├── server.ts Arranque Express: sesión, montaje de routers y middlewares, estáticos

| ├── routes/ Capa de rutas + lógica de controlador

| | ├── auth.ts /api/auth/login | logout | me

| | └── api.ts Resto de la API REST (catálogos, carga docente, horarios)

```
|   ├── lib/                  Equivalente a middlewares + helpers + servicios
|   |   ├── auth.ts           authMiddleware, roleMiddleware, cctMiddleware, ROLES
|   |   ├── db.ts             Pool mysql2, query(), withTransaction()
|   |   ├── validation.ts     Reglas de negocio puras (horas, tipo de semestre, empalmes)
|   |   └── demo-data.ts      Datos de demostración
|   └── types/express-session.d.ts  Tipado de req.session (user, activeSemesterId)
├── public/                   Frontend estático (reemplaza views/*.hbs)
|   ├── login.html + login.js   Pantalla de acceso
|   ├── index.html + app.js     SPA: Carga docente, Horarios, Catálogos
|   └── styles.css
├── tests/                    validation, schema, auth, api-catalogs (node:test vía tsx)
└── scripts/                  (vacío)
```

Capa solicitada	**Equivalente real**
src/controllers	Handlers inline en src/routes/api.ts y src/routes/auth.ts
src/routes	src/routes/*.ts (buildApiRouter, buildAuthRouter)
src/middlewares	src/lib/auth.ts
src/helpers	src/lib/validation.ts, src/lib/db.ts y helpers privados de api.ts
views/ / .hbs	public/*.html + render por JavaScript en public/app.js

2.2 Montaje de la aplicación (src/server.ts)

app.use(express.json());

app.use(session({ rolling: true, resave: false, saveUninitialized: false,

cookie: { secure: false, maxAge: 5 * 60 * 1000 } }));

app.use('/api/auth', buildAuthRouter()); // público

app.use('/api', authMiddleware, roleMiddleware, cctMiddleware, buildApiRouter());

app.use(express.static(publicDir));

app.get('*', → public/index.html);

El orden es relevante: autenticación → autorización por rol → restricción por CCT → ruta. Variables de entorno: PORT, SESSION_SECRET, DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME.

2.3 Catálogo de endpoints REST

Todos bajo /api. Las respuestas de error usan { message, code?, errors? }. Los permisos se detallan en §2.4.

Autenticación (routes/auth.ts, sin middlewares)

Método	Ruta	Entrada	Lógica
POST	/auth/login	body correo_electronico, password	Busca el usuario (JOIN tipo_usuario, centro_trabajo); exige status = 1; bcrypt.compare; session.regenerate() (anti fijación de sesión); guarda session.user y devuelve { user }. 400 sin datos, 401 credenciales inválidas/usuario inactivo
POST	/auth/logout	–	session.destroy(), limpia cookie connect.sid, 204
GET	/auth/me	–	Devuelve { user } o 401 UNAUTHENTICATED

Sistema y semestre activo

Método	Ruta	Entrada	Lógica
GET	/health	–	Público. SELECT 1; { ok, database }
GET	/semestres	id_cct (inyectado)	Lista vista_semestre ordenada por fecha_inicio DESC
GET	/semestres/:id	id	Detalle; 404 si no existe
POST	/semestres	descripcion, id_cct, tipo_semestre (P/N), fecha_inicio, fecha_fin, id_status	Si id_status = 1 desactiva el resto de semestres activos; inserta
PUT	/semestres/:id	igual que POST	Actualiza; si queda activo, desactiva los demás
DELETE	/semestres/:id	id	Borrado lógico: id_status = 2
GET	/session/semestre	–	Devuelve el semestre activo guardado en sesión
POST	/session/semestre	semesterId	Marca ese semestre como activo (id_status = 1, el resto 2) y lo guarda en session.activeSemesterId

Resolución del semestre (resolveRequestSemester): ?semesterId → body.id_semestre → session.activeSemesterId → semestre con id_status = 1 más reciente → el más reciente. Siempre acotado al CCT efectivo.

Catálogos genéricos (registerCatalogCrudRoutes)

Cada catálogo expone GET /x, GET /x/:id, POST /x, PUT /x/:id, DELETE /x/:id. Los SQL usan parámetros ? (sin concatenar valores del usuario). Con cctColumn configurado, PUT/DELETE vuelven a leer el registro y responden 403 FORBIDDEN_CCT si pertenece a otro CCT.

Ruta base	Tabla	Campos	Validación	DELETE
/centros-trabajo	centro_trabajo	CCT, Nombre	CCT ≤ 10, Nombre ≤ 100	físico
/academias	academia	nombre_academia, nombre_corto, id_especialidad, id_cct, id_status	≤ 100 / ≤ 10	lógico (id_status = 5)
/especialidades	especialidad	nombre_especialidad, nombre_corto, id_cct	≤ 100 / ≤ 20	físico
/status	status	descripcion	≤ 100	físico
/tipos-usuario	tipo_usuario	descripcion	≤ 200	físico
/usuarios	usuario	ver abajo	ver abajo	lógico (status = 5)

Usuarios: nombre (≤ 200), correo_electronico (formato y ≤ 100), tipo_usuario, status, telefono (≤ 20), id_ct, password (8–72 caracteres; obligatorio al crear, opcional al editar). La contraseña se almacena con bcrypt (12 rondas); nunca se devuelve en las consultas.

Turnos y bloques

Método	Ruta	Lógica
GET/POST/PUT/DELETE	/turnos, /turnos/:id	CRUD sobre turno.Descripcion; DELETE físico (409 si hay dependencias)
GET/POST/PUT/DELETE	/bloques, /bloques/:id	CRUD de bloque_horario (codigo_bloque, id_turno, hora_inicio, hora_fin, orden); 409 si hay horarios que lo usan

Docentes

Método	Ruta	Entrada	Lógica
GET	/docentes	–	vista_docente filtrada por CCT, ordenada por apellidos
GET	/docentes/:id	id	Detalle; 403 si es de otro CCT
POST	/docentes	RFC, Nombre, Apellido_pat, Apellido_mat, Perfil, correo_e, Telefono, horas_nombramiento, horas_descarga, id_turno, id_cct, id_status	validateDocenteHours: nombramiento 0–40, descarga 0–10, descarga ≤ nombramiento
PUT	/docentes/:id	igual	Verifica CCT del registro existente y valida horas

Método	Ruta	Entrada	Lógica
DELETE	/docentes/:id	id	Borrado lógico id_status = 5
GET	/docentes/:id/horas-disponibles	?semesterId	horas_disponibles = nombramiento − descarga; horas_asignadas = SUM(materia.horas) del semestre; horas_restantes

Materias y grupos

Método	Ruta	Entrada	Lógica
GET	/materias	?semesterId, ?grupoId, ?all=1	Sin all: solo materias con tipo_semestre y CCT del semestre; con grupoId exige que el grupo tenga el mismo semestre, tipo y CCT. Con all=1: todo el catálogo del CCT
GET	/materias/:id	id	Detalle con control de CCT
POST/PUT	/materias[/:id]	nombre, nombre_corto, semestre, horas, tipo_semestre (P/N), id_academia, id_status, id_cct	PUT valida CCT existente
DELETE	/materias/:id	id	Lógico id_status = 5

Método	Ruta	Entrada	Lógica
GET	/grupos	?semesterId, ?materiaId, ?all=1	Simétrico a materias
GET	/grupos/:id	id	Detalle con control de CCT
POST/PUT	/grupos[/:id]	grupo, semestre, id_especialidad, id_cct, id_turno, tipo_semestre	Valida P/N y CCT
DELETE	/grupos/:id	id	Físico; 409 si tiene asignaciones

Carga docente (asignacion_docente)

Método	Ruta	Entrada	Lógica
GET	/asignaciones	semesterId, docenteId y/o grupoId (al menos uno)	Consulta vista_asignacion_docente del semestre
GET	/asignaciones/:id	id	Detalle
POST	/asignaciones	id_docente, id_materia, id_grupo (+ semestre)	persistAsignacion: transacción con SELECT … FOR UPDATE sobre docente, semestre, materia y grupo; detecta duplicado y suma horas ya asignadas; ejecuta validateAsignacionDocente; INSERT. 201 { id }, 400 reglas, 409 duplicado
PUT	/asignaciones/:id	igual	Misma validación excluyendo la propia asignación

Método	Ruta	Entrada	Lógica
DELETE	/asignaciones/:id	id	Solo si el semestre está activo (403 si no) y pertenece al semestre seleccionado (409); borra y las celdas de horario caen por CASCADE

validateAsignacionDocente exige: semestre activo (id_status = 1); tipo_semestre de materia y grupo igual al del semestre; mismo número de semestre en materia y grupo; mismo CCT en semestre, docente, materia y grupo; sin duplicado; y horasAsignadas + materia.horas ≤ nombramiento − descarga.

Horario (horario_detalle)

Método	Ruta	Entrada	Lógica
GET	/catalogos	?semesterId, ?grupoId, ?materiaId	Paquete único para la SPA: activeSemester, semestres, turnos, bloques, docentes, materias, grupos, centrosTrabajo, especialidades, academias, statuses, tiposUsuario, horario
GET	/horario	?semesterId	Celdas del semestre (HORARIO_SELECT)
GET	/horario/grid	semesterId, docenteId y/o grupoId	Devuelve { activeSemester, dias, bloques, celdas }; con grupo filtra bloques por el turno del grupo
GET	/horario/:id	id	Celda individual
POST	/horario/validar	id_asignacion, dia_semana, id_bloque, id_horario?	Ejecuta las validaciones sin escribir; devuelve { validation: { valid, errors } }

Método	Ruta	Entrada	Lógica
POST	/horario	id_asignacion, dia_semana, id_bloque	Alterna celda: si está libre y válida hace INSERT (201); si ya la ocupa la misma asignación la elimina (200 removed: true); si la ocupa otra responde 409 Horario no disponible.
PUT	/horario/:id	igual	Mueve una celda tras revalidar
DELETE	/horario/:id	id	Elimina la celda (semestre debe estar activo)

validateHorarioAssignment aplica: día hábil válido; semestre activo; asignación existente y del semestre; coherencia P/N; mismo número de semestre; mismo CCT; **bloque del mismo turno que el grupo**; la materia no puede superar sus horas semanales en bloques; el docente no puede superar sus horas disponibles; **sin empalme de docente** (mismo día y bloque) y **sin empalme de grupo**. Todo se ejecuta en withTransaction con bloqueos FOR UPDATE para evitar condiciones de carrera; ER_DUP_ENTRY se traduce a 409.

2.4 Middlewares de seguridad (src/lib/auth.ts)

authMiddleware — autenticación e inactividad de 5 minutos

- Deja pasar solo /health (PUBLIC_PATHS).
- Si no existe req.session.user responde 401 { code: 'UNAUTHENTICATED' }.
- **Inactividad de 5 min:** se implementa en server.ts con cookie.maxAge = 300000 ms y rolling: true. Cada petición válida renueva la cookie y la sesión; tras 5 minutos sin actividad express-session descarta la sesión, y el siguiente request llega sin session.user → 401. El frontend (requestJson) detecta el 401 y redirige a /login.html.
- Nota operativa: no hay store configurado, por lo que se usa MemoryStore (adecuado solo para desarrollo) y cookie.secure = false; en producción con HTTPS debe cambiarse.

roleMiddleware — permisos por rol

Matriz por prefijo de ruta (ROUTE_PERMISSIONS); GET/HEAD usan read, cualquier otro método usa write. Una ruta sin entrada en la matriz queda denegada. Responde 403 { code: 'FORBIDDEN' }.

Rol (ROLES)	id
SUPERADMIN	1
ADMIN	2
CAPTURA	3
DOCENTE	4
REVISOR	5

Prefijo	Lectura	Escritura
/usuarios, /tipos-usuario, /status	Superadmin	Superadmin
/semestres, /centros-trabajo, /bloques, /turnos	Superadmin, Admin, Revisor	Superadmin
/docentes, /materias, /grupos, /academias, /especialidades, /asignaciones, /horario, /session	Superadmin, Admin, Revisor	Superadmin, Admin
/catalogos	Superadmin, Admin, Revisor	nadie

Captura (3) y Docente (4) no tienen permisos en ninguna ruta de la API.

cctMiddleware — restricción estricta por CCT (id_ct)

Aplica solo a **Administrador** y **Revisor** (CCT_RESTRICTED_ROLES); Superadministrador no se restringe.

1. Si session.user.id_ct es nulo → 403 FORBIDDEN_CCT ("sin centro de trabajo asignado").

2. Si el cliente envía id_cct en body o query y es distinto de user.id_ct → 403 FORBIDDEN_CCT.
3. Sobrescribe req.body.id_cct y req.query.id_cct con user.id_ct, de modo que las rutas aplican el filtro de forma transparente (getRequestCct).
4. En las rutas, cctMismatch() revalida el CCT de registros ya existentes antes de leer, editar o eliminar (docentes, materias, grupos, catálogos con cctColumn, horas-disponibles).

getEffectiveCct(req) expone el mismo criterio para reutilización (devuelve null para Superadmin).

2.5 Helpers y vistas (equivalente a .hbs)

No hay Handlebars: no existen registerHelper, plantillas .hbs ni partials. Se documenta el equivalente funcional.

Helpers de backend

Archivo	Helper	Función
lib/db.ts	pool, query(sql, params)	Pool mysql2 (10 conexiones) con consultas parametrizadas
lib/db.ts	withTransaction(fn)	BEGIN/COMMIT, ROLLBACK ante error, libera conexión
lib/validation.ts	getAvailableHours, validateDocenteHours, validateAsignacionDocente, validateHorarioAssignment, getScheduleCellAction, normalizeSemesterType	Reglas de negocio puras y testeables (tests/validation.test.ts)
routes/api.ts	getRequestCct, cctMismatch, getSemesterById, resolveRequestSemester, buildMateriaListFilter, buildGrupoListFilter, registerCatalogCrudRoutes, registerReadOnlyCatalogRoutes,	Utilidades internas de la capa de rutas

Archivo	Helper	Función
	persistAsignacion, validateScheduleSlot, respondScheduleError	

Páginas y "plantillas"

Archivo	Papel
public/login.html + login.js	Formulario #login-form; POST /api/auth/login; si ya hay sesión (GET /api/auth/me) redirige a /
public/index.html	Estructura de la SPA: cabecera con selector de periodo y usuario, navegación por pestañas y tres vistas
public/app.js	Estado (state), render por template strings, eventos y llamadas a la API
public/styles.css	Estilos

Vistas de index.html (equivalentes a partials por pantalla):

Vista	Contenedor	Contenido
Carga docente	#view-load	#teacher-select, resumen de horas, #assignment-form (materia, grupo, especialidad, turno), tabla #assignment-table-body
Horarios	#view-schedule	Modos grupo / docente / materias por semestre, #schedule-assignment-select, retícula #schedule-grid
Catálogos	#view-catalogs	Selector de catálogo, buscador, tabla paginada #catalog-table, acciones Nuevo/Editar/Eliminar/CSV/Imprimir, diálogo #catalog-dialog

Los "helpers" de render en app.js cumplen el papel de helpers/partials de Handlebars: escapeHtml (previene XSS en todo texto insertado), optionMarkup/fillSelect (selects), renderScheduleGrid (tabla de horario con filas de receso), renderCatalogTable, renderCatalogEditor (formulario dinámico desde catalogDefinitions), showMessage (mensajes con auto-ocultado de 5.2 s).

Control de acceso en el cliente

getCatalogAccess(key) devuelve full, readonly o none según el rol y isGlobalReadOnly() bloquea escrituras para Revisor/Captura/Docente. Es solo una mejora de UX: la autoridad real son roleMiddleware y cctMiddleware.

Conexión con el backend vía Fetch API

Toda llamada pasa por requestJson(url, options) (credentials: 'same-origin', Content-Type: application/json, manejo de 204, 401 → redirección a login y mensajes de body.message + body.errors).

Acción de la interfaz	**Llamada**
Arranque (bootstrap)	GET /api/auth/me → GET /api/semestres + GET /api/session/semestre → GET /api/catalogos?semesterId=
Cambiar periodo	GET /api/catalogos?semesterId=
Elegir docente	GET /api/asignaciones?semesterId=&docenteId= y GET /api/docentes/:id/horas-disponibles?semesterId=
Agregar asignación	POST /api/asignaciones
Cambiar grupo/docente en Horarios	GET /api/asignaciones?... y GET /api/horario/grid?...
Clic en celda de la retícula	POST /api/horario

Acción de la interfaz	Llamada
Catálogos	GET/POST/PUT/DELETE sobre endpoint de catalogDefinitions (/api/docentes, /api/materias?all=1, /api/grupos?all=1, /api/semestres, /api/academias, /api/centros-trabajo, /api/especialidades, /api/turnos, /api/bloques, /api/status, /api/tipos-usuario, /api/usuarios)
Cerrar sesión	POST /api/auth/logout

---

3. Flujo de Ejecución End-to-End

Caso: un Administrador del CCT 1 asigna el bloque M2 del lunes a una asignación docente en la vista **Horarios** (clic en una celda libre +).

Unable to render rich display

Parse error on line 26:
... (debe estar activo) H->>DB: SELECT
----------------------^
Expecting '()', 'SOLID_OPEN_ARROW', 'DOTTED_OPEN_ARROW', 'SOLID_ARROW', 'SOLID_ARROW_TOP', 'SOLID_ARROW_BOTTOM', 'STICK_ARROW_TOP', 'STICK_ARROW_BOTTOM', 'SOLID_ARROW_TOP_DOTTED', 'SOLID_ARROW_BOTTOM_DOTTED', 'STICK_ARROW_TOP_DOTTED', 'STICK_ARROW_BOTTOM_DOTTED', 'SOLID_ARROW_TOP_REVERSE', 'SOLID_ARROW_BOTTOM_REVERSE', 'STICK_ARROW_TOP_REVERSE', 'STICK_ARROW_BOTTOM_REVERSE', 'SOLID_ARROW_TOP_REVERSE_DOTTED', 'SOLID_ARROW_BOTTOM_REVERSE_DOTTED', 'STICK_ARROW_TOP_REVERSE_DOTTED', 'STICK_ARROW_BOTTOM_REVERSE_DOTTED', 'BIDIRECTIONAL_SOLID_ARROW', 'DOTTED_ARROW', 'BIDIRECTIONAL_DOTTED_ARROW', 'SOLID_CROSS', 'DOTTED_CROSS', 'SOLID_POINT', 'DOTTED_POINT', got 'NEWLINE'

For more information, see https://docs.github.com/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams#creating-mermaid-diagrams

sequenceDiagram

autonumber

actor U as Administrador (CCT 1)

participant UI as index.html + app.js

participant S as Express (server.ts)

participant A as authMiddleware

participant R as roleMiddleware

participant C as cctMiddleware

participant H as routes/api.ts POST /horario

participant V as lib/validation.ts

participant DB as MySQL

U->>UI: Clic en celda libre (data-day, data-block)

UI->>UI: toggleScheduleCell(): valida rol, periodo editable, asignación, tope de horas

UI->>S: POST /api/horario {id_semestre, id_asignacion, dia_semana:"Lunes", id_bloque:2}

S->>S: express.json() + express-session (cookie, rolling 5 min)

S->>A: ¿req.session.user existe?

A-->>S: sí (si no: 401 → login)

S->>R: prefijo /horario, método POST → write = [Superadmin, Admin]

R-->>S: ADMIN permitido (si no: 403 FORBIDDEN)

S->>C: rol restringido → fuerza body.id_cct = 1, query.id_cct = "1"

C-->>S: continúa (si envió otro CCT: 403 FORBIDDEN_CCT)

S->>H: handler

H->>DB: resolveRequestSemester → vista_semestre (id_cct = 1)

H->>H: readScheduleRequest (normaliza Miércoles→Miercoles, enteros válidos)

H->>DB: BEGIN; SELECT semestre FOR UPDATE (debe estar activo)

H->>DB: SELECT asignación + docente + materia + grupo FOR UPDATE

H->>DB: SELECT bloque; SELECT horario del semestre; SELECT ocupantes del slot

H->>V: validateHorarioAssignment(...)

V-->>H: {valid, errors} (turno, horas, empalmes docente/grupo, CCT, P/N)

alt inválido u ocupado por otra asignación

H->>DB: ROLLBACK

H-->>UI: 409 {message:"Horario no disponible.", errors:[...]}

UI->>U: showMessage(error) y reactiva el botón

else válido

H->>DB: INSERT INTO horario_detalle (id_asignacion, dia_semana, id_bloque)

H->>DB: COMMIT

H-->>UI: 201 {message:"Horario guardado correctamente.", item}

UI->>S: refreshScheduleWorkspace(): GET /api/asignaciones + GET /api/horario/grid

S-->>UI: JSON {dias, bloques, celdas}

UI->>UI: renderScheduleGrid(): celda pasa a ocupada y se actualiza "Horas asignadas: n / N"

UI->>U: Mensaje de éxito y retícula actualizada

end

Paso a paso:

1. **Interacción visual.** En #schedule-grid cada celda es un <button class="schedule-cell is-open" data-day data-block>. El listener delegado sobre el contenedor invoca toggleScheduleCell(button), que bloquea el envío si el rol es de solo lectura, el periodo no está activo, no hay asignación elegida o la materia ya completó sus horas.
2. **Petición.** requestJson('/api/horario', { method: 'POST', body }) con la cookie de sesión.
3. **Sesión e inactividad.** express-session recupera la sesión y, por rolling: true, renueva los 5 minutos de vida.
4. **authMiddleware.** Verifica session.user; sin él, 401 y el frontend redirige a login.html.
5. **roleMiddleware.** Busca el prefijo /horario; como POST es escritura, solo Superadmin y Admin pasan.
6. **cctMiddleware.** Para el Administrador fuerza id_cct = su id_ct, rechazando con 403 cualquier intento de usar otro centro.

7. **Controlador.** Resuelve el semestre (acotado al CCT), normaliza el payload y abre una transacción (withTransaction).
8. **Consultas MySQL.** Se bloquean con FOR UPDATE el semestre, la asignación (JOIN docente/materia/grupo), el bloque, el horario del semestre y los ocupantes de la celda, para que dos usuarios no puedan reservar el mismo slot.
9. **Reglas de negocio.** validateHorarioAssignment evalúa turno, horas semanales de materia y docente, empalmes, tipo P/N y CCT. La restricción UNIQUE (id_asignacion, dia_semana, id_bloque) es la última línea de defensa (ER_DUP_ENTRY → 409).
10. **Respuesta JSON.** 201 con { message, item } (o 200 con removed: true si se trataba de liberar la celda propia).
11. **Renderizado.** refreshScheduleWorkspace() vuelve a pedir asignaciones y retícula; renderScheduleGrid() reconstruye la tabla con escapeHtml, marca la celda como is-occupied, actualiza el contador de horas y muestra el mensaje con showMessage.

Anexo: observaciones detectadas durante el análisis

- POST /session/semestre actualiza semestre.id_status de **todas** las filas (no solo del CCT del usuario), por lo que cambiar de periodo activo afecta globalmente a otros centros de trabajo.
- Las rutas /semestres (POST/PUT/DELETE) no verifican CCT del registro, pero solo el Superadministrador puede escribir en ellas.
- GET /asignaciones/:id y GET /horario/:id no comparan el CCT del registro con el del usuario; el aislamiento depende del filtro por semestre en las demás rutas.
- El procedimiento sp_insertar_grupo no es usado por la API.
- SESSION_SECRET tiene un valor por defecto (sis-doc-secret); debe definirse en el entorno de producción

ESPECIFICACIÓN TÉCNICA Y PLAN DE DESARROLLO (SIS_DOC)

> ****Instrucciones para GitHub Copilot:****
> Utiliza esta especificación como directriz principal de diseño, arquitectura,
> interfaz, reglas de negocio y validación para el desarrollo del sistema web
> de horarios docentes.
>
> Toma como referencia la estructura actual de la base de datos `sis_doc`.

1. RESUMEN Y ARQUITECTURA DEL PROYECTO

El sistema ****sis_doc**** gestiona la captura estandarizada de catálogos,
la asignación de carga docente y la planeación de horarios semanales
por grupo para instituciones de Educación Media Superior en México.

* ****Backend:**** Node.js, TypeScript, Express, `mysql2/promise`, `express-session`.
* ****Frontend:**** HTML5 semántico, CSS3 Vanilla (CSS Grid/Flexbox),
  JavaScript ES6+ (Fetch API).
* ****Base de Datos:**** MySQL / MariaDB (`sis_doc`).

La arquitectura debe mantener separación clara entre:

* Presentación / Frontend.
* Rutas HTTP.

* Controladores.

* Servicios o lógica de negocio.

* Acceso a datos.

* Base de datos.

Las reglas de negocio descritas en este documento deben validarse

principalmente en el backend, independientemente de las validaciones

realizadas en el frontend.

---

2. CONTROL DE ACCESO, AUTENTICACIÓN, ROLES Y RESTRICCIÓN POR CENTRO DE TRABAJO (CCT)

El control de acceso al sistema se gestiona mediante autenticación por correo electrónico y contraseña registrados en la tabla `usuario` de la base de datos `sis_doc`.

2.1 Gestión de Sesión e Inactividad

* **Mecanismo:** Autenticación mediante `express-session` con almacenamiento seguro de estado en servidor.

* **Caducidad por Inactividad:** La sesión expirará automáticamente tras **5 minutos de inactividad** (300,000 ms).

* **Renovación:** Cualquier interacción HTTP / solicitud API restablecerá el contador del tiempo de inactividad.

* **Cierre Forzado:** Al expirar el tiempo de inactividad o pulsar "Cerrar Sesión", la sesión se destruirá y redireccionará al usuario a la pantalla de Login con la notificación correspondiente.

2.2 Restricción de Centro de Trabajo (CCT) por Usuario

* **Superadministrador (`tipo_usuario = 1`):** Acceso multi-plantel total. Puede visualizar, seleccionar y cambiar libremente entre cualquier Centro de Trabajo (`id_cct`) registrado en la base de datos.

* **Administrador (`tipo_usuario = 2`) y Revisor:** Acceso **estrictamente delimitado a su CCT de adscripción** (definido en `usuario.id_ct`).

  * En la interfaz (Frontend), el desplegable de Centro de Trabajo aparecerá preseleccionado con su CCT y **deshabilitado (`disabled`)**, impidiendo seleccionar cualquier otro plantel.

  * En el Backend, todas las consultas (`WHERE id_cct = ?`), filtrados de catálogos, carga docente y asignación de horarios se forzarán utilizando el `id_ct` almacenado en la sesión activa del usuario. Cualquier intento de enviar o manipular un `id_cct` distinto en la petición responderá con un error HTTP `403 Forbidden`.

2.3 Matriz de Permisos por Rol (`tipo_usuario`)

| Módulo / Catálogo | Superadministrador (`tipo_usuario = 1`) | Administrador (`tipo_usuario = 2`) | Revisor (Solo Lectura) | Captura / Docente (`tipo_usuario = 3 / 4`) |

| :--- | :---: | :---: | :---: | :---: |

| **Inicio de Sesión / Login** | Acceso Completo | Acceso Completo | Acceso Completo | Acceso Completo |

| **Selección de Centro de Trabajo (CCT)** | **Todos los CCT** (Libre Selección) | **Fijo a su CCT** (Sin Cambio) | **Fijo a su CCT** (Sin Cambio) | Fijo a su CCT |

| **Carga Docente y Asignación de Horarios** | Acceso Completo (Lectura/Escritura) | Acceso Completo en su CCT | **Solo Consulta en su CCT** | Sin Acceso / Solo Consulta |

| **Catálogos: Docentes, Materias, Grupos, Academias, Especialidades** | Acceso Completo | Acceso Completo en su CCT | **Solo Consulta en su CCT** | Sin Acceso |

| **Catálogos: Semestres, Centros de Trabajo, Bloques Horarios, Turnos** | Acceso Completo | Solo Consulta (Lectura) | **Solo Consulta (Lectura)** | Sin Acceso |

| **Catálogo: Estatus (`status`)** | Acceso Completo | **Sin Acceso** | **Sin Acceso** | Sin Acceso |

| **Catálogo: Tipos de Usuario (`tipo_usuario`)** | Acceso Completo | **Sin Acceso** | **Sin Acceso** | Sin Acceso |

| **Catálogo: Usuarios (`usuario`)** | Acceso Completo | **Sin Acceso** | **Sin Acceso** | Sin Acceso |

2.4 Reglas Específicas por Rol

1. **Superadministrador:** Control total sobre la visualización, creación, modificación y eliminación en todos los módulos, catálogos y centros de trabajo de la aplicación.

2. **Administrador:** Puede consultar y modificar la Carga Docente, Horarios y catálogos académicos principales únicamente dentro de su CCT asignado. Tiene acceso de solo lectura a los catálogos globales (Semestres, CCT, Bloques, Turnos) y no tiene acceso a la administración de usuarios, tipos de usuario y estatus.

3. **Revisor (Solo Lectura):** Puede consultar la Carga Docente, Horarios y catálogos académicos restringidos a su CCT asignado, pero con carácter **estrictamente de solo lectura**. No puede agregar, editar ni eliminar nada en el sistema.

4. **Captura / Docente:** Sin acceso a la gestión de catálogos generales, limitado únicamente a consultar su propia información asignada en su CCT.

2.5 Validaciones de Seguridad en Backend

1. **Middleware de Autenticación (`authMiddleware`):** Protegerá todas las rutas API (`/api/*`). Si no existe una sesión válida o se excedieron los 5 minutos de inactividad, retornará HTTP `401 Unauthorized`.

2. **Middleware de Autorización y CCT (`roleMiddleware`):** Validará en el backend que el usuario tenga permisos suficientes y que la operación corresponda estrictamente a su `id_ct` (salvo Superadministrador). Cualquier intento de modificación por un **Revisor** o acceso a datos de otro CCT por un **Administrador** será bloqueado inmediatamente con HTTP `403 Forbidden`.

3. **Adaptación de Interfaz en Frontend:** Al iniciar sesión como **Revisor**, el frontend ocultará o deshabilitará todos los botones de acción ("Nuevo", "Editar", "Eliminar", "Guardar", "Agrega"). En **Administrador** y **Revisor**, los selectores de Centro de Trabajo quedarán bloqueados en su CCT asignado.

---

3. ESTÁNDAR HOMOGÉNEO DE CAPTURA PARA TODOS LOS CATÁLOGOS

Todos los módulos de catálogos:

* `centro_trabajo`
* `usuario`
* `tipo_usuario`
* `semestre`
* `especialidad`
* `academia`
* `grupo`
* `materia`
* `docente`
* `status`
* `turno`

deben compartir un patrón de interfaz de usuario y arquitectura CRUD estrictamente homogéneo.

3.1 Estructura Visual Común de Catálogo

1. Encabezado y Barra de Búsqueda

* Desplegable de criterio de búsqueda.
* Caja de texto para búsqueda dinámica por coincidencia.

* Botón de filtro.

* Botón para limpiar búsqueda.

2. Barra de Herramientas Estándar

* **Nuevo:** limpia el formulario y abre el modo de alta.

* **Editar:** carga el registro seleccionado para modificación.

* **Eliminar / Dar de Baja:** cambia el `id_status` a Inactivo/Baja cuando corresponda o elimina físicamente el registro si no existen dependencias y la regla de negocio lo permite.

* **Guardar / Aceptar:** procesa el formulario mediante POST o PUT.

* **Cancelar:** descarta los cambios y limpia/restablece el formulario.

* **Exportar:** genera archivo `.xlsx` o `.csv` de los datos visibles.

* **Imprimir:** presenta los datos en formato para imprimir.

3. Grilla / Tabla Principal

* Columna inicial con checkbox de selección.

* Columnas ordenables.

* Filtros dinámicos.

* Paginación cuando corresponda.

* Las consultas GET utilizarán prioritariamente las vistas:

  * `vista_docente`

  * `vista_materias`

  * `vista_academias`

* `vista_grupos`

* `vista_semestre`

* `vista_asignacion_docente`

4. Formulario de Captura / Edición

Los campos deben respetar:

* nombres definidos en la base de datos;

* tipos de datos;

* longitudes máximas;

* obligatoriedad;

* claves foráneas.

Los campos relacionados mediante claves foráneas:

* `id_cct`

* `id_status`

* `id_turno`

* `id_academia`

* `id_especialidad`

* `id_semestre`

* `id_docente`

* `id_materia`

* `id_grupo`

se mostrarán mediante controles `<select>` alimentados dinámicamente

desde sus respectivos catálogos o vistas.

---

4. ESPECIFICACIÓN DE CATÁLOGOS BASE

4.1 Catálogo de Docentes

Tabla:

`docente`

Vista:

`vista_docente`

Grilla

Mostrar:

* `id_docente`
* `RFC`
* `nombre_completo`
* `Perfil`
* `correo_e`

* `Telefono`

* `horas_nombramiento`

* `horas_descarga`

* `turno`

* `centro_trabajo`

* `status`

La columna `nombre_completo` deberá mostrarse utilizando el formato:

`Apellido Paterno + Apellido Materno + Nombre`

Formulario

Datos:

* No. Control (`id_docente`)

* RFC

* Nombre

* Apellido Paterno

* Apellido Materno

* Perfil

* Correo electrónico

* Teléfono

* Horas de nombramiento

* Horas de descarga

* Turno

* Centro de trabajo (bloqueado al CCT del usuario para Administrador/Revisor)

* Estatus

Validaciones

* `horas_nombramiento` máximo 40.

* `horas_descarga` máximo 10.

* `horas_descarga` no puede ser mayor que `horas_nombramiento`.

4.2 Catálogo de Materias

Tabla:

`materia`

Vista:

`vista_materias`

Campos:

* `id_materia`

* `nombre`

* `nombre_corto`

* `semestre`

* `horas`

* `tipo_semestre`

* `id_academia`

* `id_status`

* `id_cct`

Reglas

`tipo_semestre` tendrá como valores ordinarios:

* `P` = semestre par.

* `N` = semestre impar.

Los registros históricos o especiales que actualmente utilicen:

* `X`

* `x`

deberán tratarse como registros especiales.

****Antes de implementar el filtrado final se debe definir su comportamiento:****

1. excluirlos de la carga normal;

2. permitirlos en ambos tipos de semestre; o

3. asignarles un comportamiento específico.

Copilot no debe asumir automáticamente el significado de `X/x`.

Siempre que sea posible, las materias utilizadas en asignación docente deberán tener un `id_academia` válido o null.

4.3 Catálogo de Grupos

Tabla:

`grupo`

Vista:

`vista_grupos`

Campos:

* `id_grupo`
* `semestre`
* `grupo`
* `id_especialidad`
* `id_cct`
* `id_turno`

* `tipo_semestre`

La relación principal de clasificación del grupo es:

`grupo.id_especialidad -> especialidad.id_especialidad`

No utilizar `id_academia` como clasificación del grupo.

4.4 Resto de Catálogos

centro_trabajo

* `id_cct`
* `CCT`
* `Nombre`

semestre

Vista:

`vista_semestre`

Campos:

* `id_semestre`
* `descripcion`
* `fecha_inicio`
* `fecha_fin`
* `id_cct`
* `id_status`
* `tipo_semestre`

academia

Vista:

`vista_academias`

Campos:

* `id_academia`
* `nombre_academia`
* `nombre_corto`
* `id_especialidad`
* `id_cct`
* `id_status`

especialidad

* `id_especialidad`

* `nombre_especialidad`

* `nombre_corto`

* `id_cct`

usuario

* `id_usuario`

* `nombre`

* `correo_electronico`

* `password`

* `tipo_usuario`

* `status`

* `telefono`

* `id_ct`

Catálogos simples

* `tipo_usuario`

* `turno`

* `status`

5. REGLAS DE NEGOCIO Y FLUJO DE ASIGNACIÓN DE HORARIOS

5.1 Selección del Semestre Activo

El usuario seleccionará un periodo escolar de la tabla `semestre`.

El semestre seleccionado será considerado editable únicamente cuando:

`id_status = 1`

Si el semestre tiene un status diferente de 1:

* se permitirá consultar información;
* se permitirá visualizar carga docente;
* se permitirá visualizar horarios;
* no se permitirá crear asignaciones;
* no se permitirá modificar asignaciones;
* no se permitirá agregar horarios;
* no se permitirá eliminar horarios.

Esta validación deberá realizarse obligatoriamente en el backend.

5.2 Restricción Par / Impar

El campo:

`semestre.tipo_semestre`

define el tipo de periodo activo.

Periodo Par

Si:

`tipo_semestre = 'P'`

únicamente se mostrarán materias y grupos con:

`tipo_semestre = 'P'`

correspondientes normalmente a:

* 2°
* 4°
* 6°

Periodo Impar

Si:

`tipo_semestre = 'N'`

únicamente se mostrarán materias y grupos con:

`tipo_semestre = 'N'`

correspondientes normalmente a:

* 1°
* 3°
* 5°

La validación deberá hacerse tanto al cargar los combos como al intentar guardar una asignación.

El backend no debe confiar únicamente en los filtros del frontend.

5.3 MÓDULO DE CARGA DOCENTE

Tablas / vistas principales:

* `asignacion_docente`
* `vista_asignacion_docente`
* `vista_docente`

5.3.1 Tabla Superior de Docentes

Mostrar:

* nombre completo;
* perfil;
* horas de nombramiento;
* horas de descarga;
* horas disponibles;
* horas actualmente asignadas en el semestre seleccionado.

Las horas disponibles se calculan como:

`horas_nombramiento - horas_descarga`

5.3.2 Panel de Selección

Mostrar controles para seleccionar:

* **Materia.**
* **Grupo.**

Inferencia Automática de Datos del Grupo:

* Dado que el `Turno` (`id_turno`) y la `Especialidad` (`id_especialidad`) están vinculados directamente al `grupo` en la base de datos (`vista_grupos`), **no se requiere captura manual ni selección por parte del usuario** para dichos campos[cite: 18, 19].

* Al seleccionar un `Grupo`, el sistema obtendrá automáticamente su `Turno` y `Especialidad` asociados[cite: 18, 19].

* Los campos de `Turno` y `Especialidad` se mostrarán únicamente como ****etiquetas informativas de lectura**** (**read-only** / **disabled**) dentro del formulario, actualizándose dinámicamente al cambiar el `Grupo`[cite: 19].

También mostrar:

* Contador de grupos Matutinos.

* Contador de grupos Vespertinos.

* Botón ****Agregar**** (crea el registro en `asignacion_docente` con `id_semestre`, `id_docente`, `id_materia` e `id_grupo`)[cite: 18, 19].

* ****Botón Limpiar / Resetear:**** Permite restablecer los campos seleccionados en el panel (`Materia`, `Grupo` y sus etiquetas informativas) para comenzar una nueva selección limpiamente[cite: 18, 19].

5.3.3 Vista de Asignaciones

Utilizar:

`vista_asignacion_docente`

La tabla inferior deberá mostrar al menos:

* `grupo`

* `nombre_materia`

* `horas_materia`

Las asignaciones visibles deberán corresponder exclusivamente al:

* docente seleccionado;
* `id_semestre` seleccionado.

Al pie de la columna:

`horas_materia`

mostrar la suma de las horas asignadas durante el semestre seleccionado.

5.4 VALIDACIONES AL CREAR UNA ASIGNACIÓN DOCENTE

Antes de insertar un registro en `asignacion_docente`, el backend debe realizar las siguientes validaciones.

5.4.1 Semestre Activo

El `id_semestre` seleccionado debe existir y tener:

`id_status = 1`

Si no está activo, cancelar la operación.

5.4.2 Correspondencia Par / Impar

Debe cumplirse:

`semestre.tipo_semestre = materia.tipo_semestre`

y:

`semestre.tipo_semestre = grupo.tipo_semestre`

No permitir asignaciones que violen esta regla.

5.4.3 Correspondencia del Número de Semestre

Además de ser ambos pares o impares, deberá cumplirse:

`materia.semestre = grupo.semestre`

Ejemplo inválido:

* Materia de 3° semestre.

* Grupo de 5° semestre.

Aunque ambos sean tipo `N`, la asignación no debe permitirse.

5.4.4 Compatibilidad de Centro de Trabajo

Los siguientes registros deberán pertenecer al mismo centro de trabajo:

* docente;

* materia;

* grupo;

* semestre.

Debe cumplirse:

`docente.id_cct = materia.id_cct`

`docente.id_cct = grupo.id_cct`

`docente.id_cct = semestre.id_cct`

Para Administrador y Revisor, este CCT debe coincidir estrictamente con su `usuario.id_ct`. No permitir cruces con CCT distintos.

5.4.5 Evitar Asignaciones Duplicadas

No debe existir otra asignación con la misma combinación:

* `id_semestre`
* `id_docente`
* `id_materia`
* `id_grupo`

La base de datos debe conservar una restricción UNIQUE para esta combinación.

5.4.6 Límite de Carga Docente

La suma de las horas de las materias asignadas al docente durante el semestre seleccionado no deberá superar:

`horas_nombramiento - horas_descarga`

La validación se realizará por:

* `id_docente`

* `id_semestre`

Ejemplo:

Horas nombramiento: 40

Horas descarga: 5

Máximo asignable:

35 horas.

5.4.7 Grupo Obligatorio

El flujo normal de asignación se define como:

`semestre + docente + materia + grupo`

Por lo tanto, para nuevas asignaciones:

`id_grupo`

deberá ser obligatorio desde la aplicación.

No crear asignaciones sin grupo.

5.5 MÓDULO DE HORARIOS

Tablas principales:

* `horario_detalle`
* `bloque_horario`
* `asignacion_docente`

5.5.1 Pestañas

Mostrar:

* Horario de Grupo.
* Horario de Docente.
* Materias por Semestre.

5.5.2 Selección del Grupo y Asignación Docente-Materia **[NUEVO/ACTUALIZADO]**

1. ****Selección de Grupo:**** Mostrar un combo para seleccionar el grupo, filtrado por `id_semestre` activo, `tipo_semestre` y `id_cct`. Al elegirlo, se determinará el turno (`grupo.id_turno`) para renderizar los bloques de horario correspondientes.

2. ****Selección de Asignación (Combo Docente - Materia) [NUEVO/ACTUALIZADO]:****

* Mostrar un combo con la lista de asignaciones activas en `asignacion_docente` para ese grupo y semestre.

* ****Formato del Texto en el Combo [NUEVO/ACTUALIZADO]:**** El texto de cada opción debe construirse explícitamente incluyendo las horas que le corresponden a la materia (`materia.horas`), con la siguiente estructura:

$$\text{DOCENTE} \cdot \text{MATERIA} \cdot \text{GRUPO} \cdot (\text{horas } h)$$

(Ejemplo: `CONTRERAS ROMO MARTIN · CODIFICA SOFTWARE DE SISTEMAS INFORMATICOS · 2° G · (7 h)`)

* ****Indicador Visual de Horas [NUEVO/ACTUALIZADO]:**** Al seleccionar una asignación, se mostrará dinámicamente un contador con las ****horas asignadas en la retícula / horas totales de la materia**** (Ejemplo: `Asignadas: 3 / 7 hrs`).

5.5.3 Selección de Docente / Materia

Mostrar únicamente asignaciones existentes en:

`asignacion_docente`

correspondientes a:

* grupo seleccionado;
* semestre seleccionado.

La información visible deberá incluir:

* docente;

* materia;

* horas asignadas de la materia.

5.5.4 Retícula Semanal

Columnas:

* Hora

* LUNES

* MARTES

* MIÉRCOLES

* JUEVES

* VIERNES

Las filas se cargarán desde:

`bloque_horario`

5.5.5 Bloques Matutinos

Para:

`id_turno = 1`

mostrar:

* M1 — 07:00-07:50
* M2 — 07:50-08:40
* M3 — 08:40-09:30

Receso:

09:30-09:50

Continuar con:

* M4 — 09:50-10:40
* M5 — 10:40-11:30
* M6 — 11:30-12:20
* M7 — 12:20-13:10
* M8 — 13:10-14:00

5.5.6 Bloques Vespertinos

Para:

`id_turno = 2`

mostrar:

* V1 — 14:00-14:50
* V2 — 14:50-15:40
* V3 — 15:40-16:30

Receso:

16:30-16:50

Continuar con:

* V4 — 16:50-17:40
* V5 — 17:40-18:30
* V6 — 18:30-19:20
* V7 — 19:20-20:10
* V8 — 20:10-21:00

5.6 OPERACIÓN DE LA MALLA DE HORARIOS

Al hacer clic en una celda disponible:

1. Obtener la asignación docente seleccionada.
2. Obtener `id_asignacion`.
3. Obtener `id_semestre`.
4. Obtener `id_docente`.
5. Obtener `id_grupo`.
6. Validar todas las reglas de disponibilidad y límite de horas.
7. Si no existen conflictos, insertar en `horario_detalle`:

   * `id_asignacion`
   * `dia_semana`
   * `id_bloque`

La información mostrada visualmente en la celda deberá incluir al menos:

* docente;
* materia.

5.6.1 Eliminación desde una Celda

Si la celda ya contiene exactamente la misma:

`id_asignacion`

que el usuario está intentando colocar, el nuevo clic podrá eliminar

el registro correspondiente de `horario_detalle`.

Si la celda contiene una asignación distinta:

* no eliminar automáticamente;
* mostrar mensaje:

****Horario no disponible.****

No utilizar únicamente la coincidencia del docente para decidir

la eliminación, ya que un mismo docente puede impartir diferentes materias.

5.7 VALIDACIONES BACKEND AL GUARDAR HORARIO

Todas estas validaciones deberán ejecutarse antes de insertar

`horario_detalle`.

5.7.1 Semestre Activo

La asignación deberá pertenecer al semestre seleccionado.

El semestre deberá tener:

`id_status = 1`

Si el semestre está inactivo, no permitir:

* insertar;

* modificar;

* eliminar horarios.

5.7.2 Tope de Horas de la Materia por Asignación **[NUEVO/ACTUALIZADO]**

* ****Validación de Límite de Horas de Materia [NUEVO/ACTUALIZADO]:**** Antes de registrar un nuevo bloque en `horario_detalle` para una `id_asignacion` específica, se debe contar el número total de bloques/módulos que esa `id_asignacion` ya tiene registrados en la tabla `horario_detalle` para el `id_semestre` activo.

* ****Regla de Negocio [NUEVO/ACTUALIZADO]:**** La cantidad de bloques asignados en el horario semanal ****no puede superar**** el valor de `materia.horas` correspondiente a la materia de dicha asignación[cite: 1, 20].

$$\text{Bloques Asignados} + 1 \le \text{materia.horas}$$

[cite: 1]

* ****Mensaje de Error [NUEVO/ACTUALIZADO]:**** Si la materia requiere 7 horas semanales (`materia.horas = 7`) y el usuario intenta asignar un 8° bloque para esa materia en el horario, el backend rechazará la operación y responderá con el mensaje[cite: 1, 20]:

"No se puede asignar el horario: la materia ya tiene cubiertas sus X horas semanales permitidas."

5.7.3 Sin Empalme de Docente

Un docente no puede tener más de una clase en:

* mismo `id_semestre`;
* mismo `dia_semana`;
* mismo `id_bloque`.

La combinación lógica de validación será:

`id_docente + id_semestre + dia_semana + id_bloque`

Si ya existe otra asignación, mostrar:

Horario no disponible: el docente ya tiene una clase asignada en ese horario.

5.7.4 Sin Empalme de Grupo

Un grupo no puede tener más de una clase en:

* mismo `id_semestre`;

* mismo `dia_semana`;

* mismo `id_bloque`.

La combinación lógica será:

`id_grupo + id_semestre + dia_semana + id_bloque`

Si ya existe otra asignación, mostrar:

****Horario no disponible: el grupo ya tiene una clase asignada en ese horario.****

5.7.5 Compatibilidad del Turno

El bloque horario deberá corresponder al turno del grupo.

Debe cumplirse:

`grupo.id_turno = bloque_horario.id_turno`

Ejemplo:

Un grupo Matutino no podrá utilizar bloques V1-V8.

Un grupo Vespertino no podrá utilizar bloques M1-M8.

5.7.6 Coherencia del Semestre

La asignación, materia y grupo deberán respetar el semestre activo.

Validar:

`asignacion_docente.id_semestre = semestre seleccionado`

y además:

`materia.tipo_semestre = semestre.tipo_semestre`

`grupo.tipo_semestre = semestre.tipo_semestre`

5.7.7 Número de Semestre

Validar:

`materia.semestre = grupo.semestre`

No será suficiente que ambos pertenezcan al mismo tipo P/N.

5.7.8 Centro de Trabajo

La asignación utilizada deberá pertenecer al mismo CCT del semestre

y del grupo que se está programando. Para Administrador y Revisor, deberá coincidir estrictamente con su `usuario.id_ct`.

5.8 TRANSACCIONES Y CONCURRENCIA

Las operaciones críticas de horarios deberán realizarse utilizando

transacciones SQL.

Flujo recomendado:

1. Iniciar transacción.
2. Recuperar información de `id_asignacion`.
3. Validar semestre activo.
4. Validar disponibilidad de horas de la materia (`bloques_actuales < materia.horas`)[cite: 1, 20].
5. Validar disponibilidad del docente.
6. Validar disponibilidad del grupo.

7. Validar turno.

8. Validar coherencia de semestre.

9. Insertar o eliminar `horario_detalle`.

10. Confirmar transacción.

En caso de error:

`ROLLBACK`

En caso exitoso:

`COMMIT`

Esto evita inconsistencias cuando dos usuarios intenten modificar

el horario simultáneamente.

5.9 RESPONSABILIDAD DE LAS CAPAS

Base de Datos

Responsable de:

* claves primarias;

* claves foráneas;

* restricciones UNIQUE;

* integridad referencial;

* almacenamiento persistente;

* vistas SQL;

* usuarios y contraseñas.

Backend

Responsable de:

* autenticación de usuarios y caducidad de sesión (5 min de inactividad);

* middleware de control de acceso, matriz de permisos por rol y restricción estricta por `id_ct` (salvo Superadministrador);

* semestre activo;

* permisos de modificación;

* par / impar;

* compatibilidad materia-grupo;

* compatibilidad de CCT;

* control de horas del docente;

* ****control de tope de horas por materia (`materia.horas`) [NUEVO/ACTUALIZADO]****[cite: 1, 20];

* empalme de docente;

* empalme de grupo;

* compatibilidad de turno;

* transacciones.

Frontend

Responsable de:

* formulario de inicio de sesión (Login);

* presentación según rol del usuario logueado;

* fijar y deshabilitar la selección de Centro de Trabajo (CCT) para Administrador y Revisor;

* autocompletar dinámicamente Turno y Especialidad informativos al seleccionar un Grupo en Carga Docente[cite: 18, 19];

* ****incluir el número de horas de la materia en las opciones del combo Asignación en la vista de Horarios [NUEVO/ACTUALIZADO]****[cite: 1, 20];

* ****mostrar el contador dinámico de horas asignadas vs. horas semanales permitidas [NUEVO/ACTUALIZADO]****[cite: 20];

* ocultar / deshabilitar botones de edición para perfiles restringidos o rol Revisor (solo lectura);

* selects dependientes;

* filtros;

* búsqueda;

* tablas;

* contadores;

* mensajes al usuario;

* representación de la malla;

* bloqueo visual de acciones cuando el semestre no sea editable o el rol no tenga permisos de escritura.

Las restricciones críticas siempre deben volver a validarse en backend.

---

6. CATÁLOGO DE BLOQUES DE HORARIO

Turno Matutino — id_turno = 1

* M1 — 07:00-07:50
* M2 — 07:50-08:40
* M3 — 08:40-09:30
* Receso — 09:30-09:50
* M4 — 09:50-10:40
* M5 — 10:40-11:30
* M6 — 11:30-12:20
* M7 — 12:20-13:10
* M8 — 13:10-14:00

Turno Vespertino — id_turno = 2

* V1 — 14:00-14:50
* V2 — 14:50-15:40
* V3 — 15:40-16:30
* Receso — 16:30-16:50
* V4 — 16:50-17:40
* V5 — 17:40-18:30
* V6 — 18:30-19:20
* V7 — 19:20-20:10
* V8 — 20:10-21:00

---

7. CRITERIOS GENERALES DE IMPLEMENTACIÓN

1. No duplicar datos que puedan obtenerse mediante relaciones existentes.
2. `horario_detalle` continuará almacenando únicamente:
   * `id_horario`
   * `id_asignacion`
   * `dia_semana`
   * `id_bloque`
3. No agregar `id_docente`, `id_materia`, `id_grupo` ni `id_semestre` directamente a `horario_detalle`.
4. Obtener dichos datos mediante `asignacion_docente`.
5. No confiar únicamente en validaciones JavaScript del frontend.
6. Toda regla crítica deberá validarse nuevamente en Node.js antes del INSERT, UPDATE o DELETE.
7. Utilizar consultas parametrizadas con `mysql2/promise`.
8. No concatenar datos proporcionados por el usuario directamente en SQL.
9. Mantener las rutas, controladores y lógica de acceso a datos separados.
10. Utilizar las vistas SQL para consultas de presentación cuando corresponda.
11. Los mensajes de error enviados al frontend deberán ser claros y específicos.
12. Las operaciones que afecten simultáneamente disponibilidad y horario deberán utilizar transacciones.

---

8. CRITERIO DE ÉXITO DEL MÓDULO DE AUTENTICACIÓN Y CONTROL DE ACCESO

El módulo se considerará funcionalmente correcto cuando:

1. El inicio de sesión valide credenciales directamente contra la tabla `usuario` en MySQL.

2. La sesión expire automáticamente tras **5 minutos de inactividad**.

3. El Superadministrador tenga acceso total a la consulta, edición, creación y libre selección entre cualquier Centro de Trabajo (CCT) registrado.

4. El Administrador y el Revisor queden estrictamente limitados al Centro de Trabajo registrado en su cuenta (`usuario.id_ct`), sin poder seleccionar o modificar datos de ningún otro CCT.

5. El Administrador pueda editar/agregar en la Carga Docente, Horarios y catálogos de docente, materia, grupo, academia y especialidad de su CCT, con acceso de solo lectura a los catálogos globales (semestre, CCT, bloques, turno) y bloqueo total a usuarios, tipos de usuario y estatus.

6. El **Revisor** pueda consultar la Carga Docente, Horarios y catálogos permitidos únicamente de su CCT, pero con un bloqueo **estricto a nivel de backend y frontend de cualquier acción de creación, edición o eliminación (modo 100% solo lectura)**.

7. El backend rechace con HTTP 401 o 403 cualquier petición `POST`, `PUT` o `DELETE` realizada por usuarios no autenticados, con rol **Revisor**, o que intente acceder/modificar registros pertenecientes a un CCT distinto al asignado en la sesión del usuario.

8. **En la vista de Horarios, el combo de asignación muestre explícitamente las horas correspondientes a la materia y se impida visualmente y en backend asignar más celdas/bloques en la retícula que el límite semanal permitido por la materia (`materia.horas`) [NUEVO/ACTUALIZADO]**[cite: 1, 20].